INTELLIGENT QUEUE MANAGEMENT: TECHNICAL SPECIFICATION

Get-in-Line

Real-Time Attendee Journey & Intelligent Waitlist Platform

Version	1.0
Build Plan	12 Phases over 12 weeks
Stack	Next.js · React Native · Node.js · FastAPI · PostgreSQL · Redis · RxDB
Auth	Supabase RBAC (Role-Based Access Control) + Google OAuth 2.0
Last Updated	March 2, 2026

Read this entire specification before writing a single line of code.

This document is the single source of truth for all AI coding agents. When the spec and your judgment conflict, implement the spec and add a // SPEC QUESTION: [comment].

Table of Contents

1. Product Overview

2. Tech Stack

3. Project File Structure

4. Data Models

(5-15). Phases (1-11)

16. API Reference (Full)

17. UI/UX Guidelines

18. Error Handling Standards

19. Non-Functional Requirements

20. Environmental Variables

21. Glossary

22. Agent Instructions & Constraints

1. Product Overview

Get-in-Line is a high-availability, Full-Stack Event Orchestration Platform engineered to eliminate the logistical friction inherent in high-traffic venue management. By bridging the gap between physical space constraints and the attendee journey, the system transforms chaotic queues into a streamlined, data-driven experience.

The platform utilizes a hybrid backend and local-first architecture to ensure that staff can maintain operations even in "dead zones" with zero network connectivity. For attendees, it provides a transparent, real-time interface to manage their time effectively within a venue, reducing "queue anxiety" through live data and automated notifications.

1.1 Target Users

- Venue Organizers & Staff: Need real-time visibility into room capacity, waitlist management, and crowd-flow analytics without the lag of traditional spreadsheets.
- Event Attendees: Require transparency regarding their waitlist position and the ability to autonomously manage their time and event participation.
- System Administrators: Require robust, role-based access control (RBAC) to manage large-scale data spikes and secure volunteer permissions.

1.2 Core Value Proposition

- Elevated Attendee Experience: Transparency through live wait times and vibration-alert promotions.
- Intuitive Self-Healing Queues: Automatic promotion of the next eligible attendee when a spot is vacated, ensuring 100% room utilization.
- Infrastructure Agility: Local-first architecture ensuring zero-latency check-ins in totally offline environments.
- Intelligent Forecasting: AI-driven no-show probability indicators and LSTM traffic predictions.

Platform	Hybrid: Next.js (Staff Dashboard) & React Native (Attendee Mobile).
Auth	Supabase RBAC + Google OAuth 2.0.
Persistence	PostgreSQL (Relational Integrity) + Redis (FIFO Buffer).
Offline	Local-First Architecture via RxDB and SQLite with Delta-Sync.
Environments	Development (Local) · Staging (Vercel/Fly.io) · Production (AWS/Supabase)

2. Tech Stack

To maintain the architectural integrity of Get-in-Line, all technology decisions listed below are fixed. Do not substitute libraries without adding a // ARCH DECISION: [comment] explaining the specific technical necessity.

Layer	Technology	Notes / Constraints
Frontend (Staff)	Next.js 14 + TypeScript	Dashboard for venue management. Strict mode enabled.
Mobile (Attendee)	React Native (Expo)	Cross-platform app for live queue tracking.
Backend (Real-time)	Node.js + Express	Dedicated to WebSocket handling and high-traffic queue logic.
Backend (AI/Logic)	FastAPI (Python)	Serving LSTM traffic forecasts and Monte Carlo simulations.
Database	PostgreSQL	Primary relational store for venue and user data.
In-Memory Store	Redis	Managing FIFO buffers for sub-200ms queue updates.
Offline Sync	RxDB + SQLite	Local-first architecture with delta-sync for connectivity "dead zones."
Auth	Supabase Auth + RBAC	JWT-based session management with Google OAuth 2.0 integration.
Styling	Tailwind CSS + Lucide	Consistent UI/UX language across web and mobile platforms.
Validation	Zod	Schema-first validation for all incoming API data.

3. Project File Structure

Create files in this exact structure. Do not reorganize without a // ARCH DECISION: [comment].

```
CS4485-WAITLIST-MANAGEMENT-APP-FRONTEND/
├── src/
│   ├── app/
│   │   └── components/
│   │       ├── figma/          # Specialized UI components exported or inspired by Figma designs
│   │       │   └── ImageWithFallback.tsx
│   │       └── ui/             # [45+ Collapsed] Atomic Shadcn/UI components (Buttons, Inputs, etc.)
│   ├── config/                 # Application-wide configuration and environment variable loading
│   │   └── env.ts
│   ├── hooks/                  # Custom React hooks for shared logic and mobile responsiveness
│   │   ├── use-mobile.ts
│   │   └── useWaitlistEntry.ts
│   ├── layouts/                # High-level page wrappers and navigation structures
│   │   └── AppLayout.tsx
│   ├── lib/                    # Core utility initializations (e.g., Axios instance, HTTP clients)
│   │   └── http.ts
│   ├── pages/                  # Main route components representing distinct application views
│   │   ├── DashboardPage.tsx
│   │   ├── NotFoundPage.tsx
│   │   └── VenuesPage.tsx
│   ├── services/               # Business logic for data fetching and external API communication
│   │   └── waitlistService.ts
│   ├── styles/                 # Styling system including Tailwind, Global CSS, and Typography
│   │   ├── fonts.css
│   │   ├── global.css
│   │   ├── index.css
│   │   ├── tailwind.css
│   │   └── theme.css
│   ├── types/                  # Global TypeScript interfaces and API response definitions
│   │   └── api.ts
│   ├── App.tsx                 # Main application component and routing configuration
│   ├── main.tsx                # React entry point and DOM mounting logic
│   ├── AttendeeView.tsx        # Specific view for end-user queue interactions
│   ├── CircularProgress.tsx    # UI component for visualizing wait times or loading states
│   ├── QRScanner.tsx           # Utility for camera-based check-ins and registration
│   ├── StaffDashboard.tsx      # Comprehensive management interface for venue operators
│   ├── StatusBar.tsx           # UI element for real-time system or connectivity updates
│   ├── TableGrid.tsx           # Primary data visualization for waitlist entries
│   └── utils.ts                # Shared helper functions and data formatting logic
├── .env.example                # Template for required development environment variables
├── .gitignore                  # Files and directories to be excluded from version control
├── ATTRIBUTIONS.md             # Documentation of third-party assets and credits
├── Guidelines.md               # Internal development standards and coding conventions
├── eslint.config.js            # Linter configuration for maintaining code quality
├── index.html                  # Root HTML document template
├── package.json                # Project dependencies, scripts, and metadata
├── README.md                   # High-level project overview and setup instructions
├── tsconfig.app.json           # TypeScript configuration specific to the application
├── tsconfig.json               # Base TypeScript configuration and path mapping
├── tsconfig.node.json          # TypeScript configuration for the Vite build environment
└── vite.config.ts              # Configuration for the Vite build tool and dev server
```

```
CS4485-WAITLIST-MANAGEMENT-APP-BACKEND/
├── app/                    # FastAPI AI Engine (Python)
│   ├── __init__.py         # Marks directory as a Python package
│   ├── auth.py             # AI-layer JWT/Session handling
│   ├── bootstrap.py        # System startup and dependency injection
│   ├── config.py           # Pydantic settings for API/Model keys
│   ├── errors.py           # Global exception handlers
│   ├── main.py             # FastAPI app entry point
│   ├── models.py           # LSTM/Monte Carlo math definitions
│   ├── services.py         # Business logic for queue predictions
│   └── store.py            # Local state management for Python layer
├── backend/                # Node.js/TypeScript Logic
│   ├── node_modules/       # Installed JS dependencies (usually omitted from specs)
│   └── src/                # Source code for real-time logic
├── Documentation/          # Project-level design and research docs
├── Figma Phase 2/          # Design handoff assets
├── tests/                  # Unit and integration test suites
├── .env.example            # Environment variable template
├── .gitignore              # Git exclusion rules
├── ATTRIBUTIONS.md         # Credits for libraries/assets used
├── DBDiagram.png           # Visual database schema design
├── eslint.config.js        # JS/TS linting rules
├── Guidelines.md           # Team coding standards
├── index.html              # Frontend entry HTML
├── package.json            # Node.js manifest and scripts
├── postcss.config.mjs      # CSS transformation settings
├── postgres_setup.md       # DB installation guide
├── README.md               # Project landing page
├── requirements.txt        # Python dependency list
├── schema_info.md          # Detailed table descriptions
├── schema.sql              # Raw PostgreSQL table definitions
├── tsconfig.json           # Main TypeScript configuration
├── tsconfig.app.json       # App-specific TS rules
├── tsconfig.node.json      # Vite-specific TS rules
└── vite.config.ts          # Vite build tool configuration
```

4. Data Models

4.1 Account

```sql
CREATE TABLE Account (
    UUID UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL, -- Hashed via Argon2/Bcrypt
    name TEXT NOT NULL,
    businessName TEXT,      -- Required only for role = 'BUSINESS'
    role role_type NOT NULL,
    CHECK (
        (role = 'USER' AND businessName IS NULL) OR
        (role = 'BUSINESS' AND businessName IS NOT NULL)
    )
);
```

4.2 Event

```sql
CREATE TABLE Event (
    UUID UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    BusinessAccountUUID UUID NOT NULL REFERENCES Account(UUID) ON DELETE CASCADE,
    name TEXT NOT NULL,
    eventType event_type NOT NULL,
    location TEXT,                -- CAPACITY mode only
    capacity INTEGER,             -- CAPACITY mode only
    estimatedTimePerPerson INTEGER,
    numberOfTables INTEGER,       -- TABLE mode only
    avgTableSize INTEGER,         -- TABLE mode only
    reservationDuration INTEGER, -- TABLE mode only
    noShowPolicy TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    CHECK (
        (eventType = 'CAPACITY' AND numberOfTables IS NULL AND avgTableSize IS NULL) OR
        (eventType = 'TABLE' AND location IS NULL AND capacity IS NULL)
    )
);
```

4.3 Party

```sql
CREATE TABLE Party (
    UUID UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    PartyLeaderUUID UUID NOT NULL REFERENCES Account(UUID),
    EventUUID UUID NOT NULL REFERENCES Event(UUID) ON DELETE CASCADE,
    partySize INTEGER NOT NULL,
    specialRequests TEXT
);
```

4.4 EventTable

```sql
CREATE TABLE EventTable (
    UUID UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    EventUUID UUID NOT NULL REFERENCES Event(UUID) ON DELETE CASCADE,
    PartyLeaderUUID UUID REFERENCES Account(UUID),
    PartyUUID UUID REFERENCES Party(UUID), -- Linked directly to Party table
    capacity INTEGER NOT NULL
);
```

4.5 Predictions

```sql
CREATE TABLE Predictions (
    UUID UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    EventUUID UUID NOT NULL REFERENCES Event(UUID) ON DELETE CASCADE,
    PredictedWaittime INTEGER,       -- Forecasted wait in minutes
    PredictedNoShowRate NUMERIC(5, 2), -- Phase 2: LSTM output
    PredictedFillRate NUMERIC(5, 2),   -- Phase 2: Monte Carlo simulation
    ModelVersion TEXT NOT NULL,        -- Tracking model iterations
    GeneratedAt TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

4.6 Notifications

```sql
CREATE TABLE Notifications (
    UUID UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    PartyLeaderUUID UUID NOT NULL REFERENCES Account(UUID),
    EventUUID UUID NOT NULL REFERENCES Event(UUID),
    status notification_status NOT NULL, -- 'sent' or 'failedToSend'
    sent_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

5. Phase 1 (Architecture & Research)

Feb 10 – Feb 16 · Goal: Finalize project scope and establish the offline-sync strategy for "Dead Zone" resilience.

Phase 1 is complete when: The Architecture Diagram is finalized and the tech stack (Next.js, FastAPI, RxDB) is validated for the core loop.

ID	Requirement	Acceptance Criteria
R-101	Architecture Mapping	Document flow between Node.js real-time backend and Python AI layer.
R-102	Offline Strategy	Validate RxDB + SQLite for attendee mobile devices.

Phase 1 — Build Order

1. Finalize the Guidelines.md and repository rules.
2. Create the system architecture diagram.

6. Phase 2 (Environment Setup & Database Design)

Feb 17 – Feb 23 · Goal: Initialize backend environments and deploy the core PostgreSQL schema.

ID	Requirement	Acceptance Criteria
R-201	Database Initialization	PostgreSQL schema implemented with Account and Event tables.
R-202	Environment Config	.env.example updated with Supabase and FastAPI local keys.

Phase 2 — Build Order

1. Run schema.sql to initialize local and staging DBs.
2. Setup FastAPI (Python) and Express (Node.js) boilerplates

7. Phase 3 (UI/UX Design)

Feb 24 – Mar 2 · Goal: Create high-fidelity wireframes for the Staff Dashboard and Attendee App.

ID	Requirement	Acceptance Criteria
R-301	Dashboard Mockups	Figma designs for queue management and table grid visualization.
R-302	Attendee Flow	UI for "Join Waitlist" and "Real-time Position" screens.

Phase 3 — Build Order

1. Export SVG/Component assets to src/app/components/figma/.
2. Define Tailwind theme in styles/theme.css.

8. Phase 4 (Data Sourcing & API Setup)

Mar 3 – Mar 9 · Goal: Establish core API endpoints and collect training data for AI forecasting.

ID	Requirement	Acceptance Criteria
R-401	REST API Foundations	Node.js endpoints for CRUD operations on Account and Event.
R-402	Data Sourcing	Seed database with historical traffic data for LSTM model training.

PHASE 4 — Build Order (Data Sourcing & API Setup)

Goal: Establish the communication bridge between the database and the client.

1. *Configure **Axios/HTTP client** in src/lib/http.ts to point to the backend URL.*
2. *Develop the GET /events and POST /accounts endpoints in the Node.js layer.*
3. *Create a "Seed Script" to populate the Event and Account tables with dummy data for testing.*
4. *Verify API connectivity using ApiStatusCard.tsx in the frontend.*

9. Phase 5 (Core Waitlist Logic)

Mar 10 – Mar 16 · Goal: Implement functional logic for joining, queuing, and updating attendee status.

ID	Requirement	Acceptance Criteria
R-501	Queue Join Loop	waitlistService.ts handles party creation and entry validation.
R-502	Status Management	API updates Party status between WAITING, NOTIFIED, and CHECKED_IN.

PHASE 5 — Build Order (Core Waitlist Logic)

Goal: Implement the "Join-to-Check-in" functional loop.

1. Build the logic in waitlistService.ts to handle POST /party/join.
2. Implement SQL triggers or backend logic to update Event.currentLoad when a party joins.
3. Develop the "Notify" logic to change a party status to notified and log it in the Notifications table.
4. Test the CHECK constraints in the Event table to ensure "Table" vs "Capacity" logic is enforced.

10. Phase 6 (Frontend Integration)

Mar 17 – Mar 23 · Goal: Connect the React/Next.js UI to the core waitlist logic for a Functional Alpha.

ID	Requirement	Acceptance Criteria
R-601	Live Dashboard	StaffDashboard.tsx displays real-time updates from the Party table.
R-602	Attendee Tracker	AttendeeView.tsx shows live position and estimated wait time.

PHASE 6 — Build Order (Frontend Integration - MVP)

Goal: Connect the UI components to the live data services.

1. Connect TableGrid.tsx and StaffDashboard.tsx to the live EventTable and Party data.
2. Implement the QRScanner.tsx logic to trigger a "Check-in" API call upon successful scan.
3. Link AttendeeView.tsx to the useWaitlistEntry hook to provide real-time position updates.
4. Apply global styles from styles/global.css and theme.css to all functional pages.

11. Phase 7 (AI Integration)

Mar 24 – Mar 30 · Goal: Deploy FastAPI models to provide predictive wait times and no-show rates.

ID	Requirement	Acceptance Criteria
R-701	LSTM Wait Times	Python service populates PredictedWaittime in the Predictions table.
R-702	No-Show Analytics	Probability scores calculated and displayed on the Staff Dashboard.

PHASE 7 — Build Order (AI Integration - Predictive)

Goal: Activate the FastAPI intelligence layer.

1. Initialize the **FastAPI** environment using app/bootstrap.py.
2. Implement the LSTM forecasting model in app/models.py to calculate PredictedWaittime.
3. Create a background worker in app/services.py that periodically updates the Predictions table.
4. Expose the prediction data to the frontend through a dedicated AI-dashboard endpoint.

12. Phase 8 (Stress Testing & QA)

Mar 31 – Apr 6 · Goal: Conduct high-volume event simulations and resolve performance bottlenecks.

ID	Requirement	Acceptance Criteria
R-801	Concurrent User Load	System handles 100+ simultaneous "Join Waitlist" requests without WebSocket drops.
R-802	AI Inference Latency	LSTM prediction updates complete in <500ms under load.
R-803	Edge Case Handling	"Dead Zone" simulation: Data correctly syncs via RxDB when connectivity is restored.

13. Phase 9 (Beta Deployment)

Apr 7 – Apr 13 · Goal: Release Beta version for peer testing and initial bug logging.

ID	Requirement	Acceptance Criteria
R-901	Staging Deployment	Frontend (Vercel) and Backend (Fly.io/Heroku) successfully linked to production PostgreSQL.
R-902	Bug Reporting Pipeline	Integration of a logging tool or GitHub Issues flow for Beta testers.
R-903	Real-world UX Audit	Completion of 5 "Join to Check-in" loops by non-developer testers.

14. Phase 10 (Polishing & Final Demo)

Apr 14 – Apr 20 · Goal: UI/UX refinement and final deployment to production hosting.

ID	Requirement	Acceptance Criteria
R-1001	Visual Polish	All components strictly follow the Phase 2 Figma guidelines and typography.
R-1002	Error State UI	User-friendly notifications for failed SMS or network timeouts.
R-1003	Performance Tuning	Frontend Lighthouse score > 90 for performance and accessibility.

15. Phase 11 (Handover & Documentation)

Apr 21 – May 1 · Goal: Finalize GitHub handover, README, and project documentation.

ID	Requirement	Acceptance Criteria
R-1101	Documentation Audit	README.md, ATTRIBUTIONS.md, and Guidelines.md fully updated.
R-1102	Repository Cleanup	Removal of all unused branches and sensitive environment variables from history.
R-1103	Final Presentation	Video demo showcasing the full "Intelligent Forecasting" loop.

Phase 8–11 — Build Order

1. Execute K6 or Jmeter stress tests on the Node.js/FastAPI endpoints.
2. Deploy the "Beta" branch to the production hosting provider.

3. Conduct a "Final Polish" pass on all src/app/components/ui/ components.
4. Generate the final SQL export and API documentation for the project handover.

16. API Reference (Full)

This section defines the API contract for the **Get-in-Line** system. The architecture supports three distinct venue types, a dual-queue system (Reservations + Waitlist), and an offline-first synchronization strategy.

16.1 Base URLs & Authentication

- **Production:** https://api.waitlist-app.com/v1
- **Development:** http://localhost:8000/v1

Authentication Methods:

1. **JWT Bearer Token:** Required for all Client (Mobile/Web) requests.
    - Authorization: Bearer <token>
2. **API Key:** Required for Service-to-Service (Node.js to FastAPI) communication.
    - X-API-Key: <api-key>

16.2 Event Types & Configurations

The API enforces strict validation based on the eventType to ensure data integrity for different venue layouts.

Type	Use Case	Key Logic
OUTDOOR	Festivals, Markets	Manual capacity tracking; no assigned seating.
INDOOR_TABLES	Restaurants, Cafes	Grid-based (4-column) table management; smart assignment.
INDOOR_SEATED	Theaters, Stadiums	Individual seat assignment; capacity = total seats.

16.3 Core Endpoints

POST /events

- **Goal:** Initialize a venue waitlist.
- **Request Body:**

JSON

```
{
  "name": "Summer Festival",
  "eventType": "OUTDOOR",
  "maxCapacity": 5000,
  "startTime": "2026-06-15T14:00:00Z",
  "offlineEnabled": true
}
```

- **Logic:** Auto-generates tables/seats if type is INDOOR_*.

POST /events/{eventId}/waitlist

- **Goal:** Add a guest (Attendee) to the queue.
- **Status Flow:** QUEUED → NOTIFIED → SEATED.
- **Special Request Parsing:** * "Table 5" → Specific assignment.
    - "Near Sarah" → Proximity logic.
- **Response:** Includes position and estimatedWait (derived from AI Prediction table).

POST /sync

- **Goal:** Resolve offline data conflicts when a device regains connectivity.
- **Strategy:** SERVER_WINS.
- **Payload:** Array of operations (CREATE, UPDATE) with local timestamps.

16.4 Error Handling Contract

All errors follow a standardized structure to assist in debugging during the **Phase 8-10 Testing** period.

Status	Error Code	Description
409	ALREADY_EXISTS	Guest is already active in this queue.
409	NO_CAPACITY	Event has reached maxCapacity.
403	FORBIDDEN	Staff-only endpoint accessed by a USER role.

16.5 Implementation Build Order (API Reference)

1. **Phase 4:** Establish POST /auth/login and GET /events to unblock frontend development.
2. **Phase 5:** Implement the QUEUED → SEATED state machine logic.
3. **Phase 7:** Integrate the FastAPI prediction engine to populate the estimatedWait field.

4. **Phase 8:** Stress test the /sync endpoint with simulated high-latency connections.

17. UI/UX Guidelines

All User Interfaces (Staff Dashboard and Attendee Mobile View) are built using **Tailwind CSS** integrated via **Vite** to ensure high-performance, utility-first styling. To maintain a consistent design system across the **11-phase development cycle**, the following rules apply:

- **Utility-First Styling:** Direct Tailwind classes are preferred for all layouts to eliminate the need for custom .css stylesheets.
- **Component Consistency:** Common UI elements (Buttons, Inputs, Dialogs) should leverage a standardized component library to ensure accessibility and keyboard navigability.

- **Inline Style Restriction:** Inline styles are strictly prohibited except in rare cases where Tailwind's configuration cannot achieve the required dynamic result (e.g., complex AI data visualizations).

17.1 Layout & Navigation

The application uses a **State-Based View Management** system. Instead of traditional URL routing, the StaffDashboard manages its internal layout via a currentPage state ('home' | 'waitlist' | 'capacity'), allowing for a seamless, "App-like" experience on a single page.

- **View Encapsulation:**
  - **Home View:** A grid-based "Event Hub" showing all active events with status indicators (Active, Paused) and real-time occupancy counters.
  - **Waitlist Management:** A dual-mode list view (waitlist vs. reservation) that allows staff to promote, seat, or mark guests as no-shows.
  - **Capacity/Queue View:** A specialized layout for **Line Management** featuring progress bars for queue capacity and auto-calculated wait time metrics.
- **Navigation Shell:**
  - **Header:** A persistent black-themed header providing a global Menu toggle, Logout action, and dynamic sub-titles that reflect the current sub-page.
  - **Mobile-First Constraint:** The entire layout is constrained to max-w-md and centered, optimized for staff members using handheld tablets or smartphones while moving around a venue.

17.2 Component Rules & Feedback

The project utilizes a library-driven approach to maintain visual consistency:

- **Iconography:** lucide-react is used for all semantic actions (e.g., Plus for creation, UserX for no-shows, Clock for wait times).

- **Real-time Feedback:** * **Toasts:** The sonner library handles transient notifications, providing context-aware success/error messages (e.g., "Sarah Johnson seated at Table 4").
    - **Sync Indicators:** A StatusBar component provides a visual cue (isSyncing) to inform the user when local state changes are being persisted or synchronized.
- **Destructive Actions:** Significant data deletions (e.g., "Clear all tables") utilize native confirm() dialogs to prevent accidental data loss, ensuring a safety check for high-consequence operations.
- **Data Visualization:** Custom CircularProgress and linear progress bars are used to represent occupancy percentages, using color-coding (Green/Amber/Red) to signal venue density levels.

17.3 Color System (Light Mode)

Token	**Value (Hex/OKLCH)**	**Usage**
primary	#030213	Main brand color, primary actions, and headers.

Token	Value (Hex/OKLCH)	Usage
accent	#E9EBEF	Secondary highlights and hover states.
background	#FFFFFF	Main page and card background.
destructive	#D4183D	Error states, delete actions, and danger alerts.
muted	#ECECF0	Background for subtle elements or disabled states.
muted-foreground	#717182	Secondary text, placeholders, and labels.
border	rgba(0,0,0,0.1)	Dividers, input outlines, and component boundaries.

18. Error Handling Standards

10.1 API Error Response Format

Every API error returns this exact JSON shape to ensure the frontend can parse it consistently.

```
{
  "code": "string",       // Machine-readable (e.g., 'RESOURCE_NOT_FOUND')
  "message": "string",    // Human-readable message safe for users
  "details": "any",       // Optional: field-level validation errors
  "timestamp": "string", // ISO 8601 UTC timestamp
  "requestId": "string"  // Unique ID for log tracking (e.g., 'req-1709...')
}
```

10.2 Standard Error Codes

Based on your error.ts logic and the Waitlist API structure, these are the primary error codes your system handles:

Code	Status	Description
RESOURCE_NOT_FOUND	404	The requested endpoint or database record does not exist.
VALIDATION_ERROR	400	Input failed validation (typically used with Zod). Includes details.
UNAUTHORIZED	401	Missing or invalid authentication credentials.
FORBIDDEN	403	Authenticated user lacks permission for the specific resource.

Code	Status	Description
INTERNAL_ERROR	500	Unexpected server error. Stacks are logged but never exposed to the UI.

10.3 Middleware Implementation

Your server.ts routes all traffic through two key safety nets at the very end of the stack:

- **notFound Middleware:** Catches any requests to routes that aren't defined (e.g., /v1/unknown) and throws a 404 RESOURCE_NOT_FOUND.
- **errorHandler Middleware:** The final catch-all. It checks if an error is an instance of your ApiError class. If it is, it returns the custom status and code; otherwise, it defaults to a 500 INTERNAL_ERROR.

19. Non-functional Requirements

Based on your documentation for **Section 11: Non-Functional Requirements** and the **Non-Functional Specifications** for the Waitlist API, here is the formalized breakdown for your project.

11.1 Security

- **Data Protection**: Passwords must be hashed using **bcrypt** with a cost factor of **12**; plain text storage is strictly prohibited.
- **Session Management**: Access tokens expire in **15 minutes**, while refresh tokens expire in **7 days**. Refresh token hashes are stored in the database to allow for server-side invalidation.
- **Authorization**: All API routes must verify that the authenticated userId matches the resource owner before any read or write operation to prevent **IDOR (Insecure Direct Object Reference)** vulnerabilities.
- **Access Control**: **Role-Based Access Control (RBAC)** is implemented to ensure restricted actions (e.g., deleting venue data) are only available to authorized personnel.
- **Rate Limiting**:
  - Login endpoint: Maximum **10 requests per minute** per IP.
  - Coach messages: Maximum **30 per user per day**.
- **CORS**: In production, requests are only allowed from the designated FRONTEND_URL.

11.2 Performance & Scalability

- **Scalability**: The system uses **partitioned PostgreSQL tables** to handle massive spikes in attendance.
- **Latency**:

    - API response times must be **< 300ms** for all non-AI endpoints under normal load.
    - WebSocket updates must propagate to all clients in **under 200ms**.
- **Reliability**: A **99.9% uptime** is required for the notification service to prevent venue bottlenecks.
- **Database Efficiency**: Analytics queries must never perform full table scans; they must always filter on an **indexed userId** first.
- **Frontend Speed**: The dashboard **LCP (Largest Contentful Paint)** must be **< 2.5 seconds** on a standard connection, and the initial frontend bundle must be **< 500KB gzipped**.

11.3 Accessibility

- **Navigation**: All interactive elements must be keyboard-navigable with **visible focus rings**.
- **Standards**: The application must maintain **WCAG 2.1 AA** minimum text contrast ratios throughout the UI.
- **Assistive Tech**: All form inputs must have associated labels, and color must **never** be the sole means of conveying information.

11.4 Code Quality & Testing

- **Type Safety**: The project uses **TypeScript strict mode** with no implicit any allowed anywhere. All API handler functions must have explicit return types.
- **Testing**: A minimum test coverage of **70%** is required for utility functions and API routes.
- **Environment Management**: Three distinct environments are maintained: **development** (local), **staging** (Vercel preview with a separate DB), and **production**.

20. Environment Variables

Create .env.example with all keys and no values. Config must be validated via Zod at backend startup — fail fast if any required variable is missing.

Variable	Description	Example / Range
PORT	The port the Express server listens on.	8000
NODE_ENV	Defines the current runtime environment.	development, staging, production

21. Glossary

NEED TO BE UPDATED

22. Agent Instructions & Constraints

Read this section before writing any code. These rules override any other instinct or default behavior.

14.1 Phase Gates

This specification is divided into 11 sequential phases. Each phase acts as a strict "Hard Gate"—you must not proceed to the next phase until every requirement of the current phase is met, verified, and (where applicable) passes all tests with >70% coverage.

- **Phase 1 Gate:** Complete initial boilerplate, environment setup, and base error handling.
- **Phase 2 Gate:** All Phase 1 requirements must pass; database schema and migrations for core entities must be finalized.
- **Phase 3 Gate:** All Phase 2 requirements must pass; complete Auth system (JWT/OAuth) and user management.
- **Phase 4 Gate:** All Phase 3 requirements must pass; core API CRUD operations for primary resources must be functional.
- **Phase 5 Gate:** All Phase 4 requirements must pass; implement real-time features (WebSockets) and state sync.
- **Phase 6 Gate:** All Phase 5 requirements must pass; frontend dashboard and core UI components (shadcn) must be interactive.
- **Phase 7 Gate:** All Phase 6 requirements must pass; background jobs (crons) and automated notification services must be active.
- **Phase 8 Gate:** All Phase 7 requirements must pass; implement advanced analytics and indexed reporting queries.
- **Phase 9 Gate:** All Phase 8 requirements must pass; full Accessibility (WCAG 2.1 AA) and focus ring audit complete.
- **Phase 10 Gate:** All Phase 9 requirements must pass; perform final Security hardening (Rate limiting, CORS, and IDOR checks).

- **Phase 11 Gate:** All Phase 10 requirements must pass; complete End-to-End (E2E) testing and production environment staging.

14.2 Build Order Within Each Phase

Even within these 11 phases, you must maintain the internal build order to prevent architectural drift:

1. **DB Schema/Migrations**
2. **API Routes & Business Logic**
3. **Unit & Integration Tests**
4. **Frontend Components & State**
5. **Frontend/E2E Tests**

14.3 Security Rules (Non-Negotiable)

- **Secret Management:** Never hardcode secrets, user IDs, or URLs. All configuration must come from environment variables.
- **Data Isolation:** All database queries must be scoped to the authenticated user's userId. Never allow a query to leak data across users.
- **Error Exposure:** Never expose stack traces to users. All production errors must return generic, safe messages.
- **Key Protection:** Any API keys (e.g., Anthropic) must never appear in frontend code, browser requests, or logs.

14.4 Handling Gaps

If this specification does not cover a situation you encounter, apply these rules in order:

1. Check if a similar pattern is already established elsewhere in the spec and follow it.
2. Choose the approach that is most consistent with the existing architecture.
3. Choose the simpler of two reasonable approaches.
4. Leave a comment in the code explaining your choice.